DASH-01: Dashboard Fundamentals

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 1 of 7 | **Created:** March 2026 | **Last Updated:** 10/26/2026

Overview

Dashboards are the primary visualization layer in Dynatrace, turning raw observability data into actionable insights. This notebook covers the distinction between dashboards and notebooks, when to use each, the architecture of Dynatrace dashboards (tiles, Markdown header tiles, variables), and core design principles that make dashboards effective. Whether you are building your first dashboard or refining an existing library, these fundamentals provide the foundation for every notebook in this series.

Dashboards Failing Validation Do Not Load (SaaS 1.344, tightened in 1.346)

> **A dashboard that fails validation does not load (SaaS 1.344; stricter rules in SaaS 1.346).** SaaS 1.344 was published 07/27/2026 with a staged tenant rollout from 07/29/2026, and SaaS 1.346 restates the rule with stricter validation. Confirm which version your tenant runs — enforcement arrives with the version. Before 1.344, a dashboard whose payload failed validation still loaded and surfaced validation warnings; from 1.344 it does not load at all until fixed.
>
> Dynatrace names dashboards **created externally via API or by AI tooling** as the most affected population. If you build dashboards only in the UI, this changes little — the UI emits shapes it can render. If you deploy dashboards as code (Documents API, Monaco, Terraform) or generate them with a script or an assistant, treat the warning state as a grace period rather than a supported state: deploy to a non-production tenant, open the dashboard in the Dashboards app, and confirm zero validation warnings before merging. **DASH-07 §5** carries the full gate, including why a `2xx` from the Documents API is not render confirmation.

> **Sources:** What's new in Dynatrace SaaS 1.346 (DT docs) — *"Starting with Dynatrace version 1.346, Dynatrace applies stricter validation rules to dashboards and won't display dashboards that fail validation until you fix them."*

New Dashboard Building Blocks

Recent platform changes make several dashboard patterns simpler:

1. **OneAgent primary fields/tags as top-level fields** (OneAgent 1.333+, Latest Dynatrace) on metrics, spans, logs, events and entities. Filters and `by:` groupings can dispatch on `dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`, and customer-defined `primary_tags.<key>` directly — no more `parse(content, ...)` in dashboard tile queries. Particularly impactful for executive dashboards (DASH-03) that need cost-by-business-unit views.
2. **Ownership by team** — split by owning team with primary Grail tags, not a node attribute. There is no `ownership.*` field on Smartscape nodes: on a validation tenant (09/24/2026) `getNodeField(id, "ownership.team")` returned null for 35 of 35 services while `getNodeField(id, "name")` resolved all 35, and `dt.semantic_dictionary.fields` has no field containing `ownership`. Primary tags do reach metrics: *"Dynatrace enriches all derived signals (service metrics, Davis events, and problems) with the same tags"* (Primary Grail fields and tags (DT docs)). So a tile can split by `primary_tags.team` directly:

```
dql timeseries requests = sum(dt.service.request.count), by:{primary_tags.team}, from:-2h
```

The split is only as good as the tag — where no `team` primary tag is set, every series lands in one null group. For notifying the owning team from a workflow, the Ownership app's *Get owners* action *"Retrieves owners from entities and team identifiers"* (Actions for Ownership (DT docs)).

3. **`dt.service.name`** (semantic dictionary: `stable`, *"equal to the Smartscape service node name"*) sits beside the OTel `service.name` — split service-level tiles by name without joining through entity enrichment.

Table of Contents

1. Dashboards vs Notebooks
2. Dashboard Architecture
3. Design Principles
4. Dashboard Creation Workflow
5. Your First Dashboard Query
6. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards
Permissions	`storage:logs:read`, `storage:metrics:read`, `storage:events:read`
Access	Dashboard creation privileges (`document:documents:write`)
Data	At least 1 hour of ingested logs and metrics

1. Dashboards vs Notebooks

Dynatrace provides two primary visualization tools, each designed for different use cases.

Feature	Dashboard	Notebook
Purpose	Ongoing monitoring and reporting	Ad-hoc exploration and analysis
Layout	Grid-based tile arrangement	Sequential, document-style cells
Audience	Teams, stakeholders, wall screens	Individual analysts and engineers

Feature	Dashboard	Notebook
Interactivity	Variables, filters, drill-down links	Inline DQL editing, iterative queries
Refresh	Auto-refresh (configurable interval)	Manual execution per cell
Sharing	Access for all, user/group shares, share links; scheduled delivery via Workflows (DASH-07 §3)	Share as document or export
Persistence	Saved as Dynatrace documents	Saved as Dynatrace documents

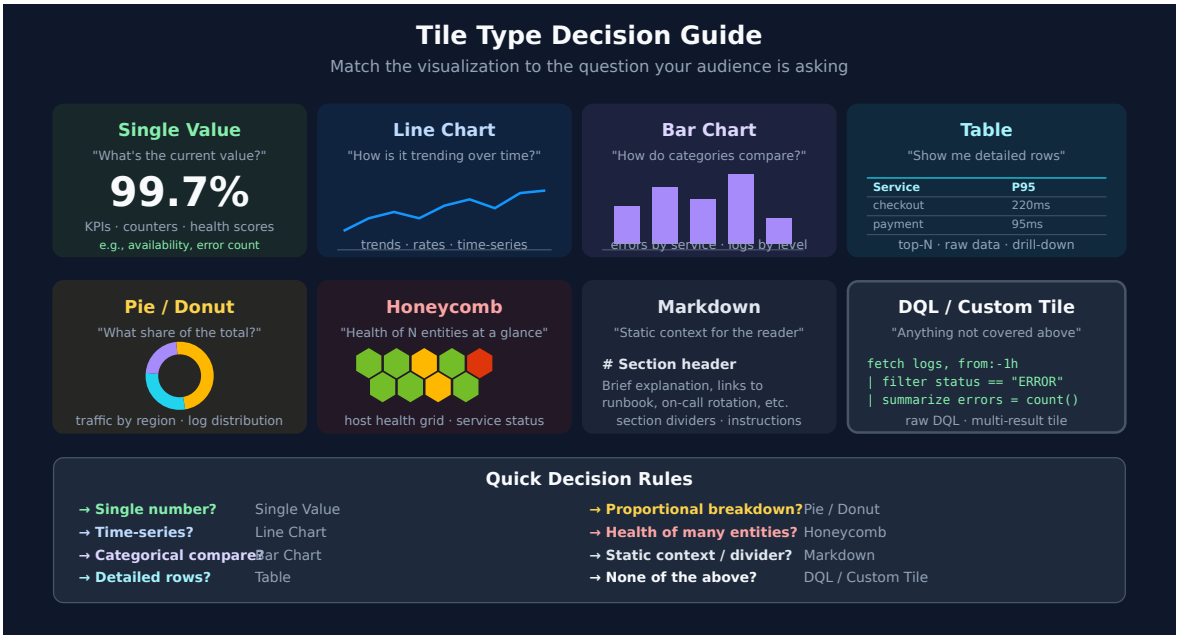
When to Use a Dashboard

- **Continuous monitoring** — service health, SLA tracking, infrastructure overview
- **Team visibility** — wall screens, shared views across roles
- **Stakeholder reporting** — executive summaries, weekly status
- **Alerting context** — a landing page linked from alert notifications

When to Use a Notebook

- **Incident investigation** — iterating on queries to find root cause
- **Data exploration** — discovering patterns in new data sources
- **Knowledge sharing** — documented analysis with explanatory text
- **Training** — step-by-step DQL tutorials (like this series)

2. Dashboard Architecture



A Dynatrace dashboard is built from three things: tiles, the way you group them, and variables.

Tiles

Tiles are the individual visualization units. Each tile contains a single DQL query (or metric expression) and renders as a specific chart type.

Tile Type	Best For	Example
Single value	KPIs, counters, health scores	Error count, availability %
Line chart	Trends over time	CPU usage, request rate
Bar chart	Categorical comparisons	Errors by service, logs by level
Table	Detailed data, top-N lists	Top 10 slowest endpoints
Pie / Donut	Proportional breakdowns	Traffic by region
Honeycomb	Entity health at a glance	Host health grid
Markdown	Static text, instructions	Section headers, links

Grouping Tiles

A dashboard is a free-form grid of tiles; it has no collapsible "section" container (sections belong to Notebooks). Group related tiles by placing them together under a **Markdown tile** used as a header — for example, a "Service Health" header above the service tiles and an "Infrastructure" header above the host tiles.

Variables

Variables make dashboards dynamic. A single dashboard template can serve multiple teams by letting users pick values — a service, an environment, a namespace — from variables populated by a DQL query, a list, code or free text. The timeframe comes from the dashboard's own timeframe selector, not a variable. Variables are covered in depth in **DASH-06**.

Release Radar (April 2026): New Tile Types and Authoring Controls

Recent additions to the dashboard authoring surface:

Addition	What it does	Where it helps
Treemap visualization	New tile type for hierarchical data — area encodes magnitude so dominant categories pop out (e.g. requests per service, grouped by Kubernetes namespace)	Spotting the big contributors in nested categories where a bar chart would need many rows
Row marker coloring (tables)	Color-coded row markers visually group related rows while keeping the table readable	Operations tables (DASH-04) where rows belong to tiers, teams, or severities
Centralized tile indicator controls	Show/hide warnings, descriptions, and custom timeframes across *all* tiles from one dashboard-level control	Cleaning up a dense dashboard for a wall screen vs. an editing session
Dashboard variables for dynamic coloring	Variables can drive coloring and threshold conditions, so visual rules stay in sync with the selected environment/team	Reusable templates — see **DASH-06**

Addition	What it does	Where it helps
Direct image upload in Markdown	Built-in image library: upload images directly inside Markdown tiles across Dashboards, Notebooks, and Launcher	Embedding logos, legends, or architecture snippets without an external host

Add **Treemap** to the tile-type vocabulary alongside the chart types in the table above. It is the right choice when you want *share-of-total within a hierarchy* at a glance, rather than precise per-category values.

3. Design Principles

Effective dashboards follow consistent design principles regardless of the audience.

Clarity

- **One purpose per dashboard** — avoid "everything" dashboards that try to serve every audience
- **Descriptive tile titles** — a tile titled "Error Rate (%) — Checkout Service" is immediately understandable; "Query 1" is not
- **Consistent time ranges** — all tiles should use the dashboard time selector unless there is a specific reason for a fixed range

Audience Awareness

- **Executives** need 3-5 high-level KPIs, not 30 detailed charts
- **Operations** teams need real-time status and alert context
- **Engineers** need drill-down capability and raw data access

Design for your audience first, then consider the data.

Actionability

Every tile should answer a question or trigger an action:

Tile Shows	Action
Error rate above threshold	Investigate in notebook or traces

Tile Shows	Action
SLA breach trending	Escalate to engineering
Deployment marker + latency spike	Correlate with release
Host CPU saturated	Scale or investigate process

If a tile does not lead to a decision, consider removing it.

Layout Best Practices

- **Top-left anchor** — place the most critical KPI in the top-left (first thing the eye hits)
- **Z-pattern reading** — arrange tiles so the dashboard reads left-to-right, top-to-bottom
- **Group related tiles** — place them together under a Markdown header tile
- **Limit tile count** — aim for 8-12 tiles per dashboard; more than 15 creates cognitive overload

4. Dashboard Creation Workflow

Follow this structured approach when building a new dashboard:

Step 1: Define the Audience and Purpose

Answer these questions before opening the dashboard editor:

- **Who** will view this dashboard? (role, technical level)
- **What** decisions will it support?
- **When** will it be viewed? (real-time wall screen, weekly review, incident response)
- **How often** should data refresh?

Step 2: Identify Key Metrics

Select 5-10 metrics that directly answer the audience's questions. Validate each query in a notebook first.

Step 3: Prototype in a Notebook

Write and validate all DQL queries in a notebook before building the dashboard. This lets you iterate quickly without fighting the tile editor.

Step 4: Build the Dashboard

Transfer validated queries to dashboard tiles. Group related tiles under Markdown header tiles, following the layout principles above.

Step 5: Add Variables and Filters

Make the dashboard reusable by adding variables (DQL, List, Code or Free text — DASH-06) for the dimensions viewers filter on; leave time to the dashboard timeframe selector.

Step 6: Share and Iterate

Share with the target audience, collect feedback, and refine. A dashboard is never "done" — it evolves with the team's needs.

5. Your First Dashboard Query

Before building a dashboard, you need validated DQL queries. Let's start with a few foundational queries that appear on almost every dashboard.

Total Log Volume by Level

This query provides a quick overview of log health — useful as a bar chart tile.

```
// Log volume breakdown by severity level
fetch logs, from:-1h
| summarize log_count = count(), by:{loglevel}
| sort log_count desc
```

Active Problem Count

A single-value tile showing how many detected problems are currently active — the most common executive KPI.

```
// Count of currently active detected problems
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| summarize active_problems = countDistinct(event.id)
```

Error Log Trend Over Time

A line chart showing error frequency — ideal for spotting spikes and correlating with deployments.

```
// Error log trend over the last 6 hours
fetch logs, from:-6h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR"
| makeTimeseries error_count = count(), interval:10m
```

Host CPU Overview

A timeseries query for a line chart showing CPU usage across hosts.

```
// Average CPU usage by host over the last hour
timeseries avg_cpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
```

6. Summary and Next Steps

In this notebook you learned:

- The difference between dashboards and notebooks, and when to use each
- Dashboard architecture: tiles, grouping under Markdown header tiles, and variables
- Core design principles: clarity, audience awareness, actionability
- A structured workflow for creating dashboards
- Foundational DQL queries that appear on most dashboards

Next: In **DASH-02: Dashboard Hierarchy**, we explore the three-tier model — Executive, Operations, and Engineering dashboards — and how to design information density for each audience.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.